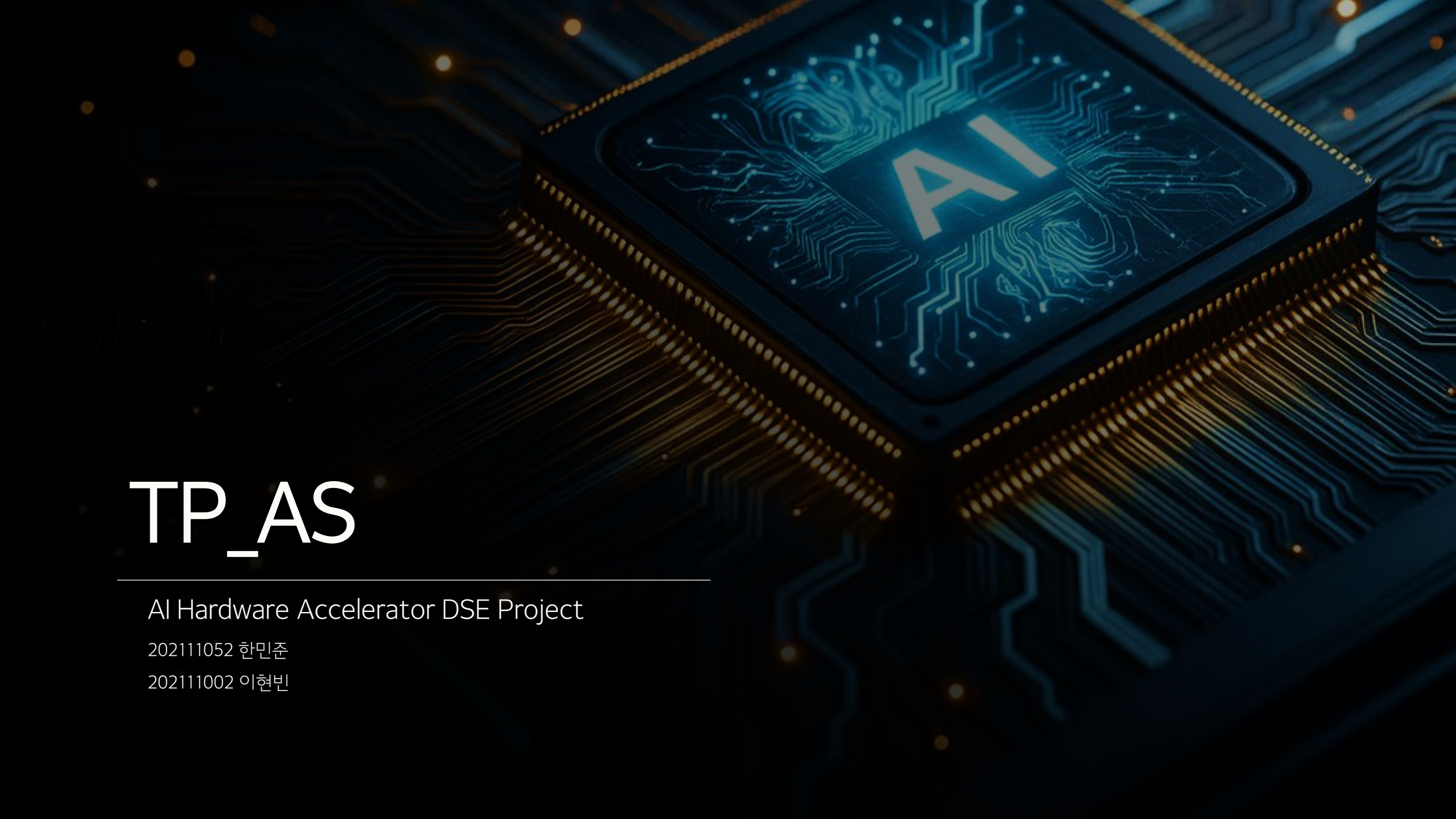


The background of the slide is a dark blue, high-tech image featuring a central square chip with the letters 'AI' in a glowing blue font. The chip is surrounded by intricate, glowing blue circuit patterns. The entire scene is set against a dark background with faint, glowing orange and blue light spots, giving it a futuristic and technological feel.

TP_AS

AI Hardware Accelerator DSE Project

202111052 한민준

202111002 이현빈

Index

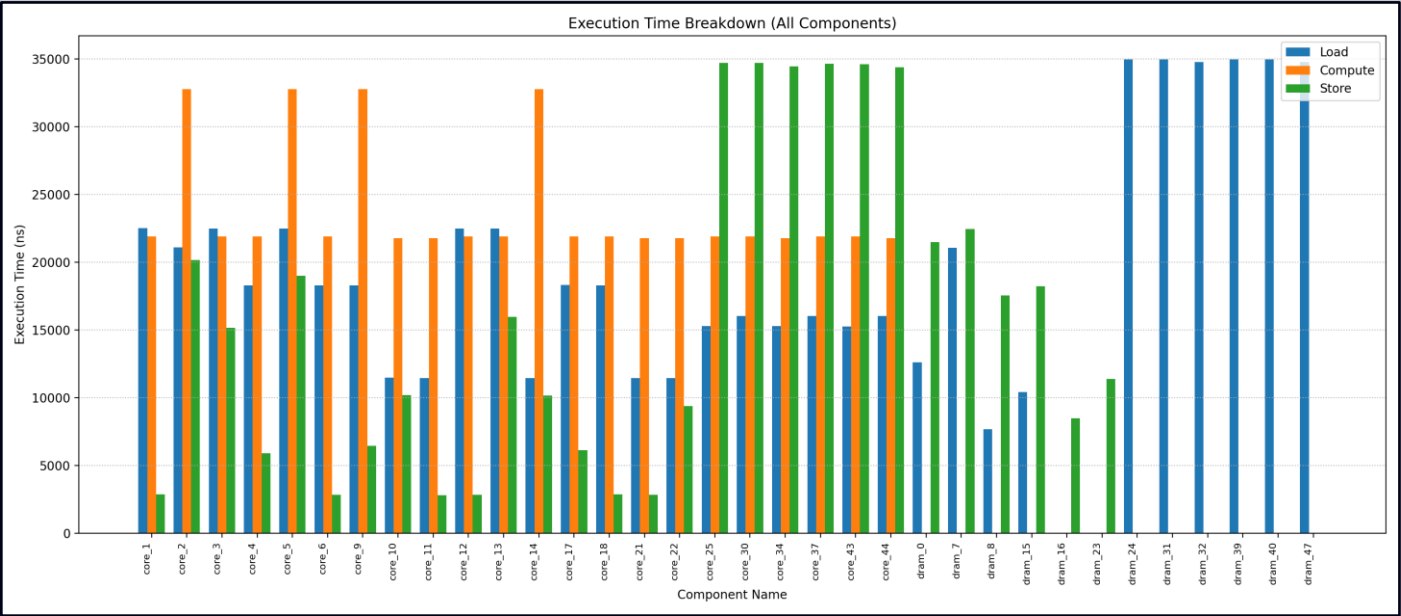
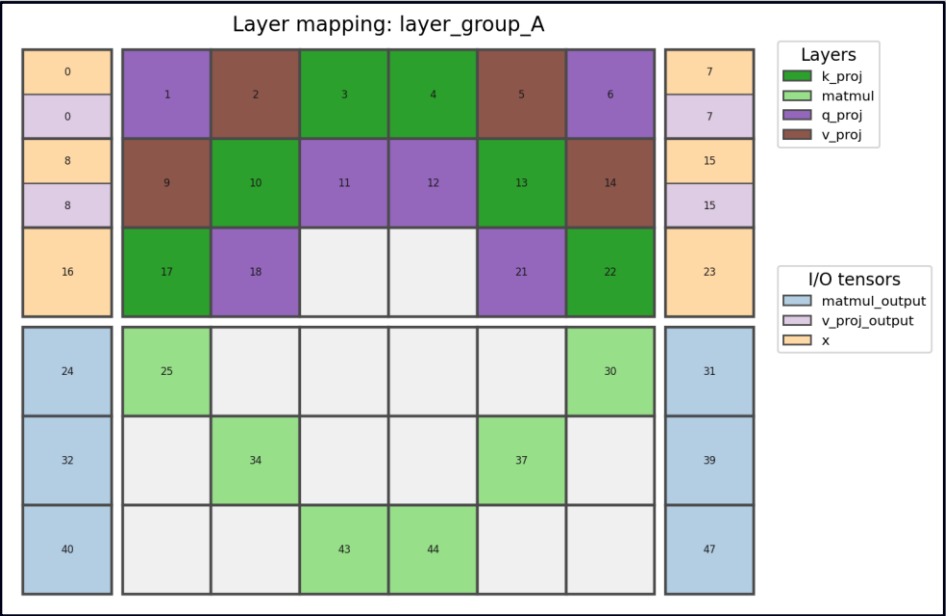
- 01 Methodology
- 02 Implementation
- 03 Results
- 04 Discussion

Methodology

Overview of the exploration strategy

Model_0

Throughput(TOPS) = 30.70



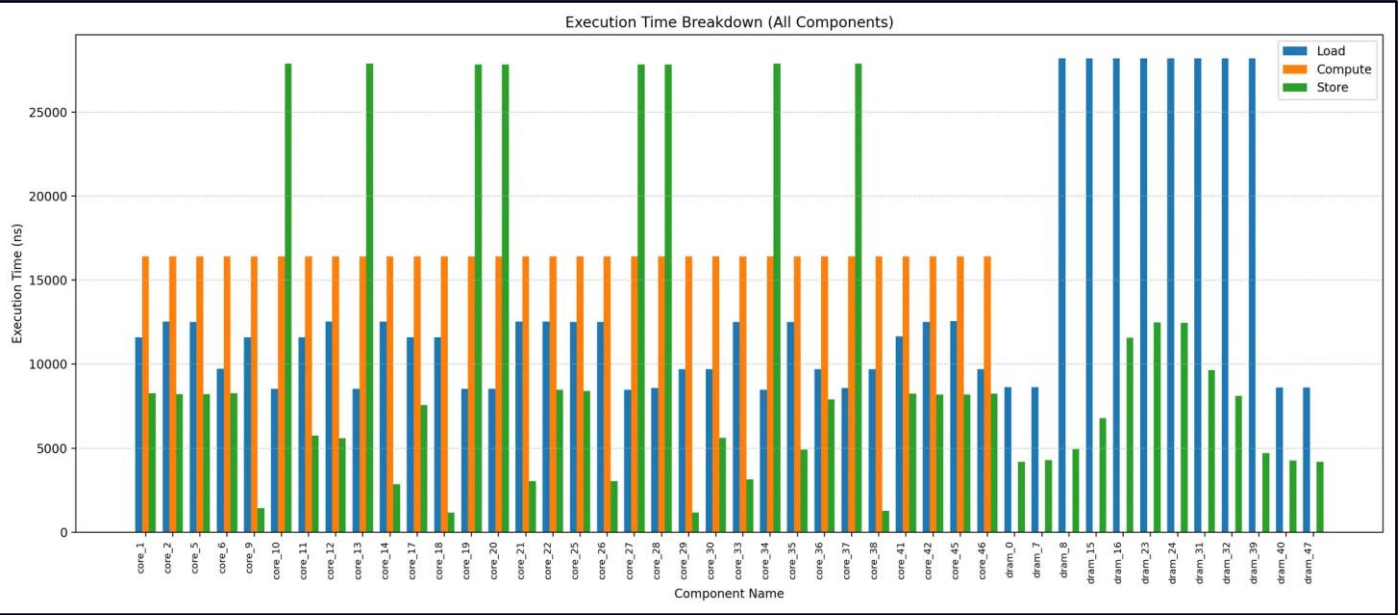
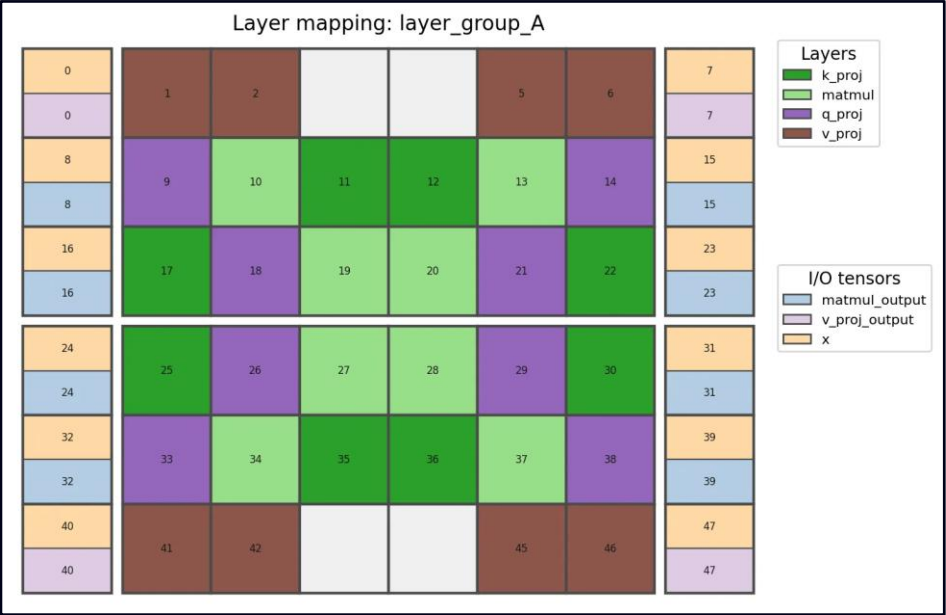
Task

Matmul output comm(store) bound
K_proj, Q_proj computing time

Solution

Hardware를 4분할

Model_1 Throughput(TOPS) = 38.09

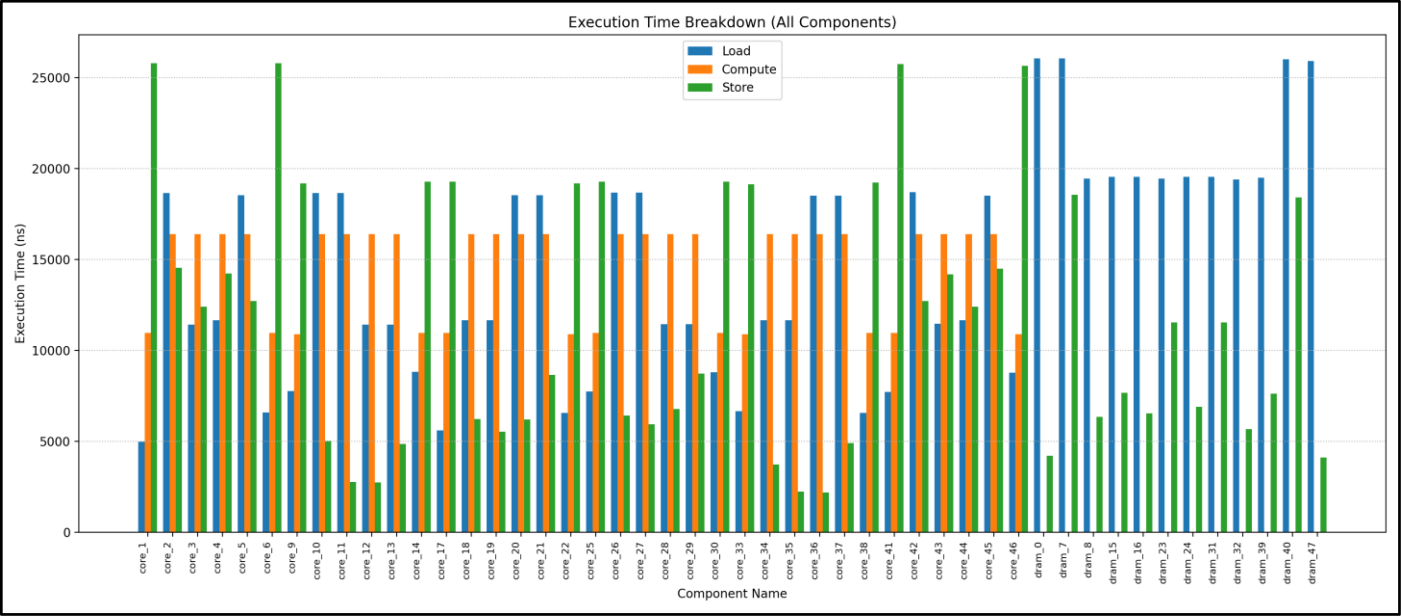
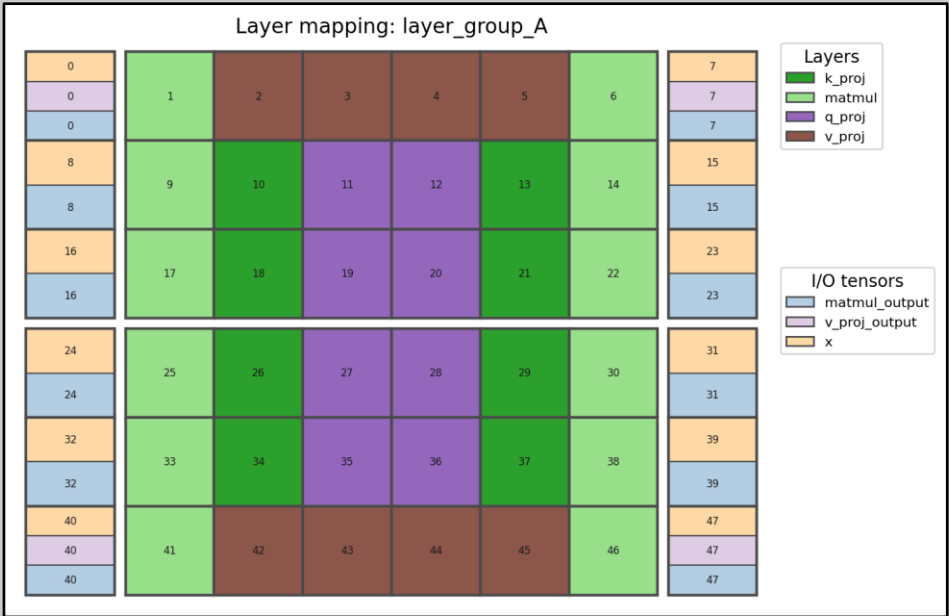


Task Matmul comm(store) bound

Solution Matmul 담당 core수 증가

Model_2 (Final model)

Throughput(TOPS) = 41.20



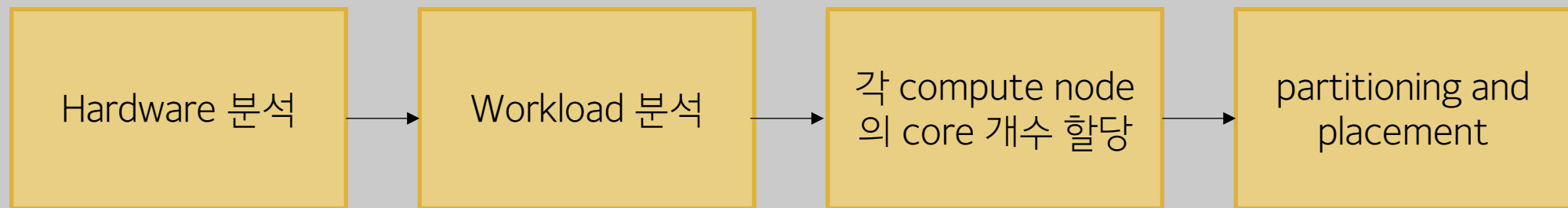
DSE strategy

Compute node중, light node를 수행하는 core의 placement에 random 특성을 부과함

Implementation

Code review

Analysis of Key Functions



parseHardware()

Function	hardware.json로부터 하드웨어 특성(core, dram 개수) 파악
----------	--

analyzeWorkload()

Function	이전 iteration의 flowID.csv, allocation.csv를 바탕으로 compute node별 load byte size와 mac_compute_count 합을 반환
----------	--

calc_symmetric_allocation()

Function	각 compute node의 "중요도" 계산을 통해, core개수 할당 성능 안정성과 통신 균형을 위해 대칭 배치 수행
Logic	1. 해당 traffic, MACs를 통해, core개수 우선순위 score 계산 2. score가 높은 compute node부터 순서대로 정렬 3. 한 compute node가 가질 수 있는 최대 core 개수 제한 설정 - 전체 코어의 약 1/3 이상 못 갖게 제한 - 항상 4의 배수로 맞춤 - aligned 제약(aligned:16 등) 있으면 더 줄어들 수 있음 4. compute node별 초기 core 개수 배정 - 최종 코어 수는 항상 4의 배수 - 최소 코어는 4개 보장 5. 전체 core를 초과하면 조정 6. core가 남으면 다시 골고루 배분 7. 최종 결과 반환 - 각 노드가 사용할 코어 개수(counts) - 노드 우선순위 리스트(sorted_ns)

execute_priority_placement()

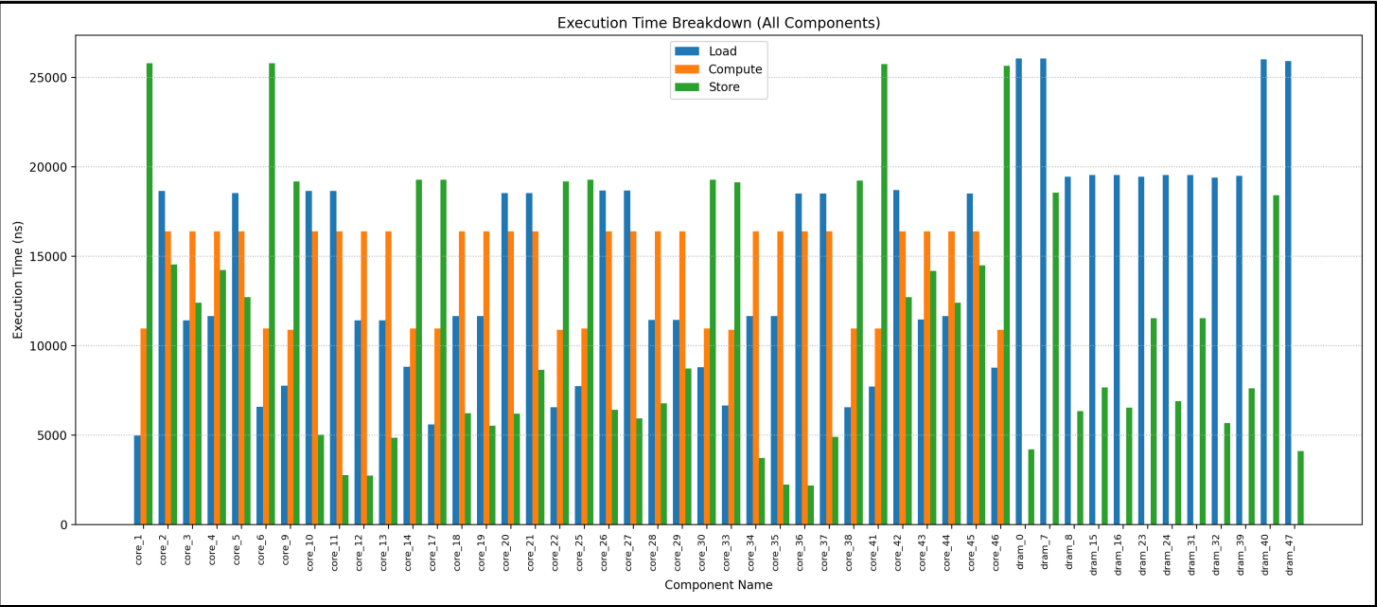
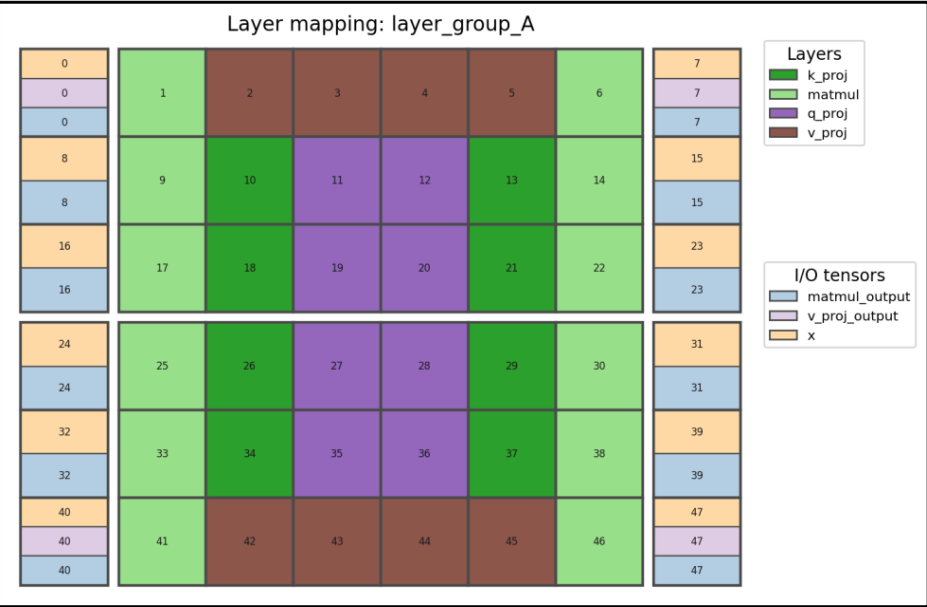
Function	placement
Logic	## execute_priority_placement --> 이전 함수의 return 값인 counts 와 sorted_ns 를 받아온다. step1 . Pooling 전체 core 를 TL, TR, BL, BR 4개로 분할하자. --> NoP link 회피 목적 --> Traffic 관리 용이 TL 의 core placement 가 완료되면, get_symmetric_cores 함수에 의해 나머지 3개의 영역은 대칭적으로 할당. Pool 생성 3종류 a) t1_iso_pool(isolation) : pole TL 기준 우측 상단(row[0]) 부터 배치하여 고립 b) t1_vertical_block_pool (Heavy) : Column[0] 부터 Topdown 방식으로 배치 (dram_router 와의 거리 최대한 가깝게) c) t1_default_pool(Light): Random 성 부여하여 다양한 조합중 최적의 Routing 찾기

execute_priority_placement()

Logic	step 2. Categorization - isolation_nodes[] : No parent, children nodes - light_nodes[] : extra - heavy_nodes[] : heavy load ,store nodes OR heavy computation step3. Placement isalated -> Heavy -> Light 순서로 로직에 따라 배치 Dram Router 는 해당 Row 에 output 을 생성하는 core(producer) 가 있다면 반드시 할당된다. => DOR XY Routing 고려.
-------	---

Results

Layer group A



Total Simulation time (ns)	26060
Total Throughput (TOPS)	41.20
Total Power Efficiency (TOPS/W)	18.54

DSE efficacy

sampling efficiency

Layer group A

Total Simulation time (ns)	26060
Total Throughput (TOPS)	41.20
Total Power Efficiency (TOPS/W)	18.54

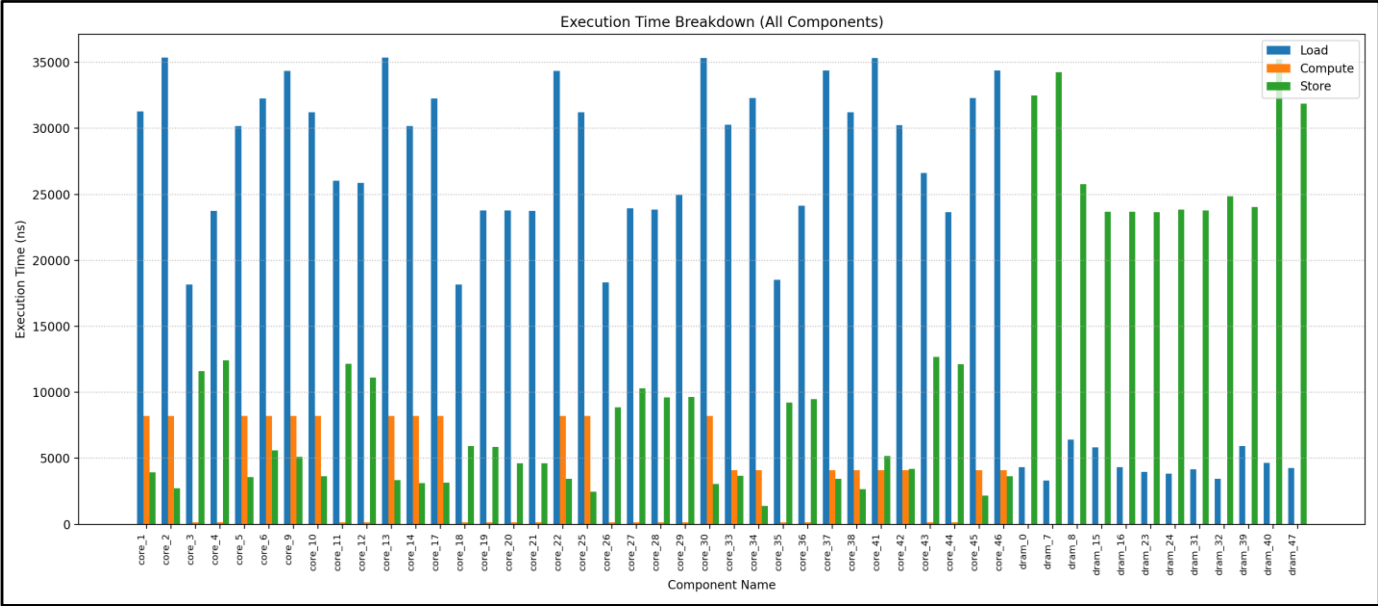
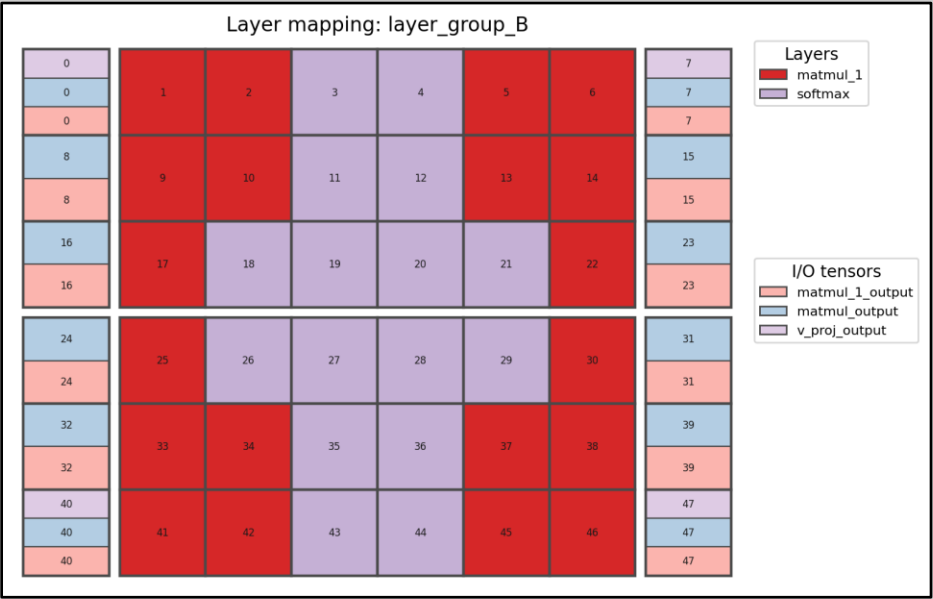
✓ $\text{throuput} / \text{iteration_count} = 41.20 / 2 = 20.6$

✓ $\text{power_efficiency} / \text{iteration_count} = 18.54 / 2 = 9.27$

DSE efficacy

general applicability

Layer group B



Total Simulation time (ns)

35350

Total Throughput (TOPS)

7.71

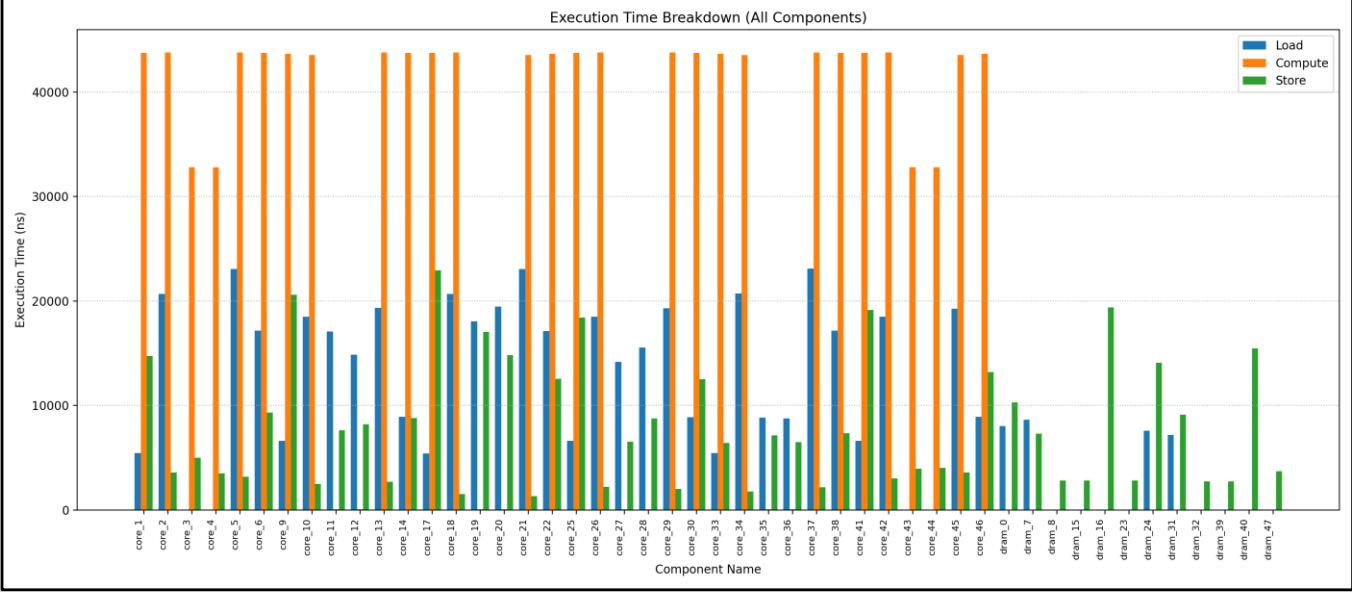
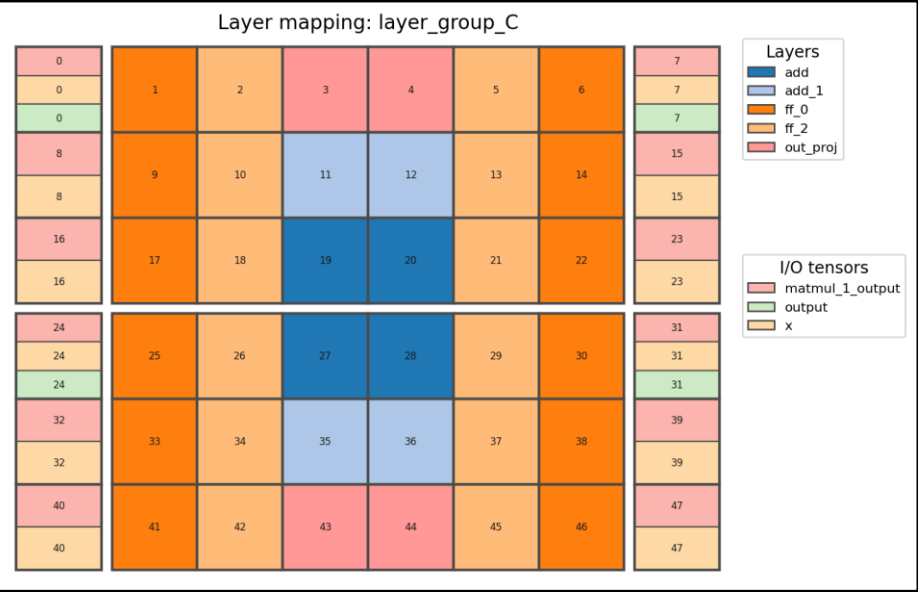
Total Power Efficiency (TOPS/W)

11.4

DSE efficacy

general applicability

Layer group C



Total Simulation time (ns)

43780

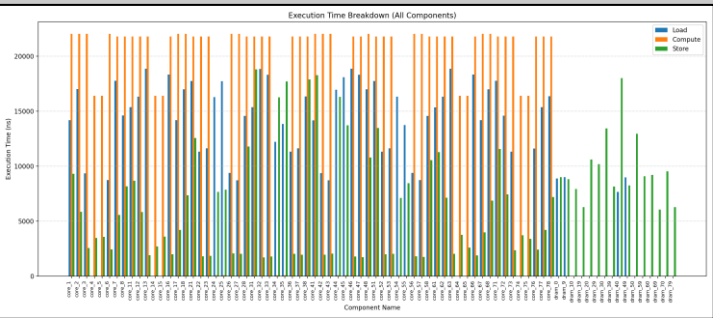
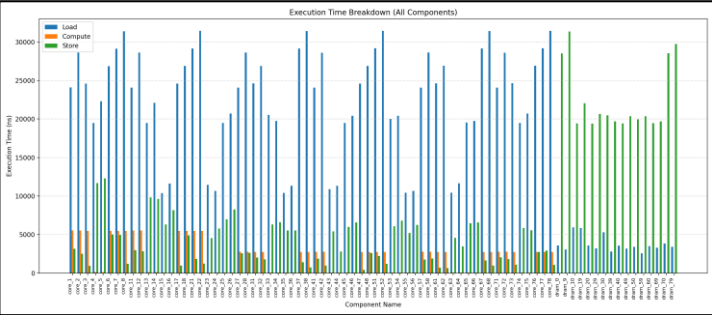
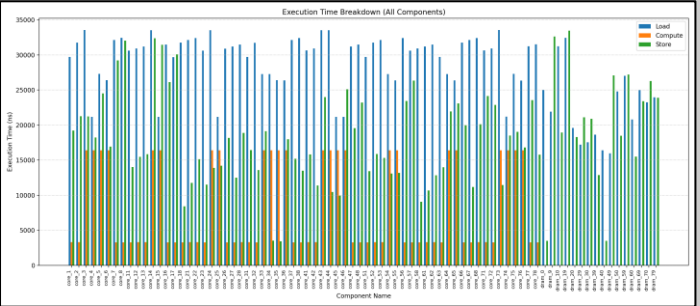
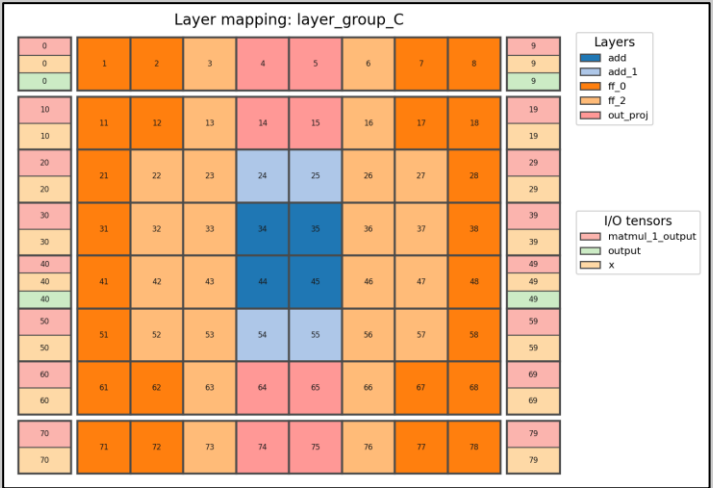
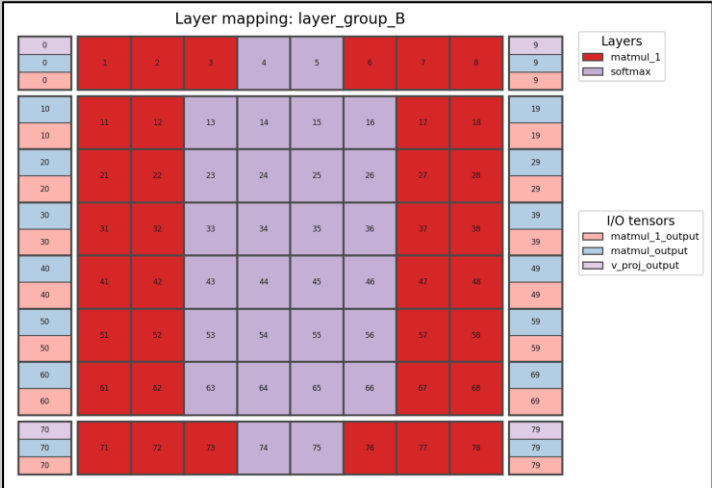
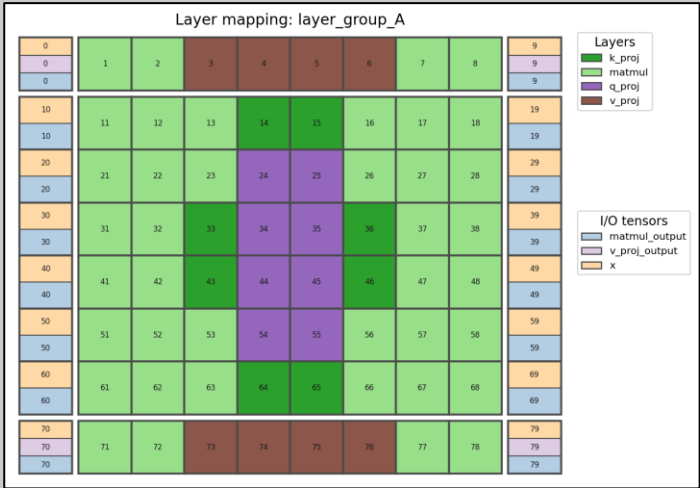
Total Throughput (TOPS)

55.2

Total Power Efficiency (TOPS/W)

21.82

64core_16dram



	Layer A	Layer B	Layer C
Total Simulation time (ns)	33540	31430	22020
Total Throughput (TOPS)	32.01	8.67	109.7
Total Power Efficiency (TOPS/W)	18.40	11.06	20.66

Dicussion

성능 개선 과정에서의 인사이트

Hardware를 4분할 함으로써, 데이터 flow control logic을 간소화
core간의 NoP link를 피할 수 있음

제안 방법의 장점과 한계

장점 : inference과정에서의 logic이 매우 자세하고, 다양한 workload, hardware에 대한 범용성이 좋음

한계 : 알고리즘 상 이전 iteration을 토대로 학습을 거듭하여 더 좋은 방향으로 거듭해나가는 과정은 random성에 의존함

